

Задание 1 - Алгоритм шифрования RSA по блокам

Перед тем, как прислать решение, обязательно прочтите все дополнительные требования к решениям, которые описаны в отдельном(ых) файле(ах), а также файл с дедлайном.

Цель задания

- Научиться шифровать произвольные сообщения.
- Реализовать алгоритм RSA для шифрования и расшифровки произвольных сообщений с разбиением на блоки.

Предварительные требования к заданию

В ходе лабораторных занятий у вас должен быть реализован алгоритм RSA для шифрования чисел со всеми вспомогательными алгоритмами:

- использовать встроенную функцию `pow` для быстрого возведения в степень по модулю и вычисления обратного элемента по модулю,
- использовать библиотечную функцию для проверки числа на простоту (например, `isprime` из библиотеки `sympy`).

Гарантируется, что число n имеет байтовую длину больше 1.

Задачи

1. Нужно реализовать шифрование блоками в RSA (все способы). [Подробнее ниже](#).

Должна быть возможность шифрования произвольных:

а. строки без применения `pickle`,

б. объекта в программе (например, список, словарь, ...) с помощью `pickle`.

Ещё раз повторю: данные могут быть произвольными!

2. Вся функциональность должна быть реализована с помощью функций, интерфейс (консольный или GUI) писать не нужно.
3. Тщательно вручную протестировать реализованный алгоритм.

Шифрование блоками в RSA

Генерация ключей

Задаётся число k – количество бит. Далее с помощью собственной реализации алгоритма Миллера-Рабина генерируются два неравных простых числа p и q битности k . Далее выполняется оставшаяся часть генерации ключа в RSA, в которой получается открытый ключ – пара чисел (e, n) , где n имеет битовую длину $m = 2k$ (или

почти $2k$), а также вычисляется закрытая экспонента d и определяется закрытый ключ (d, n) .

Общий алгоритм шифрования блоками с помощью RSA

Этапы шифрования:

1. Преобразование в массив байтов. Текст/**произвольный объект** в программе преобразуется в массив байтов V .
 - а. текст кодируется с помощью кодировки,
 - б. объект сериализуется с применением модуля pickle.**
2. Разбиение на подгруппы. Далее этот массив V разбивается на группы/подмассивы байтов v_i размером $byte_size(n) - 1$, то есть на один байт короче, чем байтовая длина числа n . Таким образом, v_i представляет собой массив байтов.
 - а. **Последняя группа: во всех алгоритмах вы сами должны придумать такой способ работы с последним блоком, чтобы при расшифровке вы смогли получить изначальную последнюю группу и, в следствие этого, исходное сообщение.** Для этого можете выполнять любые действия как перед шагом, так и после этапа 2.
3. Обработка групп перед шифрованием. Здесь могут выполняться различные действия над v_i в зависимости от выбранного [способа](#).
4. Шифрование. Затем каждая группа шифруется следующим образом. Массив v_i преобразуется в число h_i , которое затем шифруется в число r_i уже реализованным вами алгоритмом RSA для чисел.
5. Преобразование зашифрованного числа в массив байтов. Число r_i далее преобразуется в массив байтов u_i различными [способами](#).
6. Объединение результата. В конце все u_i объединяются в один общий массив байтов U , который и является зашифрованным сообщением. Таким образом, у вас должен получаться массив байтов, а не список, кортеж, строка и так далее.

Для расшифровки нужно выполнить этот алгоритм в обратном порядке.

Далее будут описаны уже конкретные алгоритмы шифрования, которые вам нужно будет реализовать. Они отличаются лишь этапом 3 «Обработка групп перед шифрованием» и 5 «Преобразование зашифрованного числа в массив байтов».

«Обработка групп перед шифрованием»: 3 способа

Способ 1 – без обработки

Никакой обработки. Ничего делать не нужно:) Но это не значит, что будет легко:)

Способ 2 – со случайным старшим байтом

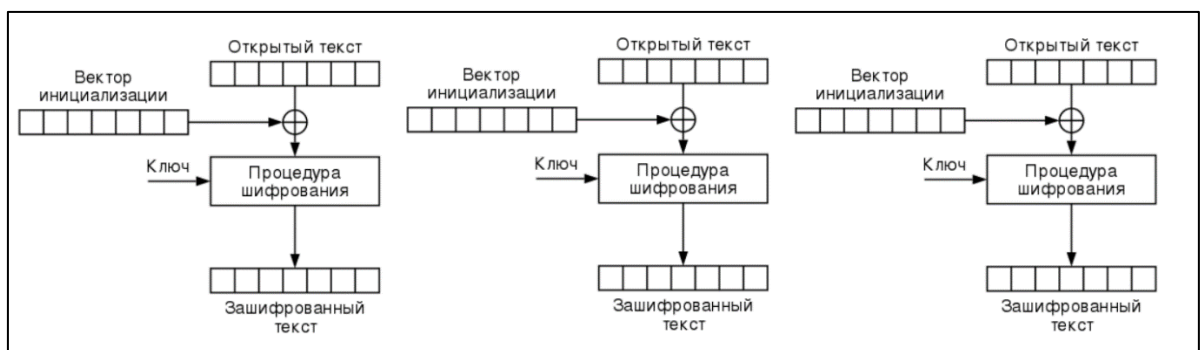
3. Обработка групп перед шифрованием. Каждая группа v_i обрабатывается следующим образом.

- a. Все группы кроме последней: к полученному массиву байтов v_i случайно добавляется в начало старший байт таким образом, чтобы получился массив, который представляет собой число $h < n$.
 - i. генерация этого байта должна быть осуществлена без применения циклов/рекурсий и тому подобного.
- b. Последняя группа также должна заполняться случайными байтами до размера $byte_size(n)$. Вы сами должны придумать этот алгоритм так, чтобы при расшифровке вы смогли получить изначальную последнюю группу и, в следствие этого, исходное сообщение.

Способ 3 – аналог режима сцепления блоков (CBC)

3. Обработка групп перед шифрованием. Реализовать аналог режима сцепления блоков (CBC) для блочных алгоритмов шифрования.

a. Шифрование



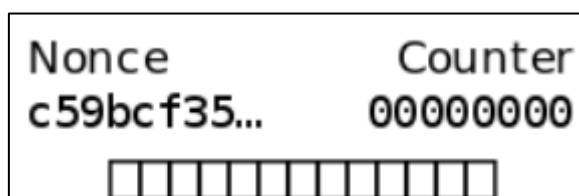
- b. Вам нужно сгенерировать вектор инициализации – случайный массив байтов, который до этого не использовался.
 - i. Для этого вам необходимо вести учёт уже использованных векторов инициализации.
 - ii. Размер вектора инициализации вы должны выбрать самостоятельно.
- c. Перед шифрованием вектор инициализации XOR'ится с блоками v_i .

Вектор инициализации нужно добавить к зашифрованному сообщению после выполнения этапа 6 «Объединение результата» для получения одного общего массива байтов, который будет содержать зашифрованное сообщение и вектор инициализации.

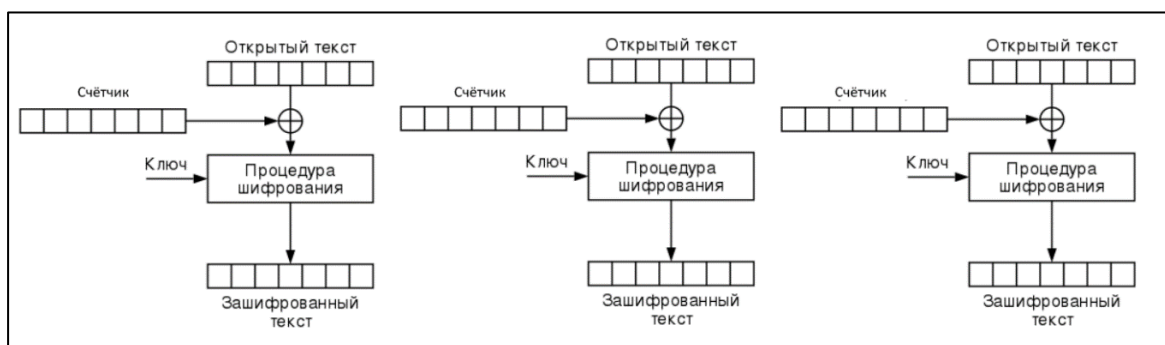
Способ 4 – аналог режима счётчика (Counter)

3. Обработка групп перед шифрованием. Реализовать аналог режима счётчика для блочных алгоритмов шифрования.

а. Счётчик



б. Шифрование



- с. Всё, как в предыдущем способе, но в качестве вектора используете counter: массив байтов, половина которого представляет собой случайные данные, а вторая половина – счётчик – номер блока.
- д. Вектор перед шифрованием XOR'ится с блоками v_i .

Counter нужно добавить к зашифрованному сообщению после выполнения этапа 6 «Объединение результата» для получения одного общего массива байтов, который будет содержать зашифрованное сообщение и counter.

Преобразование зашифрованного числа в массив байтов: 2 способа

Способ 1 – с упаковкой в массивы одинаковой длины

5. Преобразование зашифрованного числа в массив байтов. Число r_i преобразуется в массив байтов u_i размера $byte_size(n)$.

Способ 2 – с добавлением длины блока

5. Преобразование зашифрованного числа в массив байтов. Число r_i преобразуется в массив байтов u'_i минимальной длины для хранения этого числа, затем перед

u'_i добавляется количество занимаемых байтов, в результате чего получаем массив байтов u_i .

Шифрование блоками: резюме

Таким образом, вам нужно реализовать $4 \times 2 = 8$ вариантов шифрования и расшифровки:

- 4 способа «Обработки групп перед шифрованием»;
- 2 способа «Преобразование зашифрованного числа в массив байтов».

Кроме того, для каждого варианта будет 2 разных исходных объекта шифрования: произвольные текст и объект.